

SENTINEL: Insider Threat Detection Based on Multi-Timescale User Behavior Interaction Graph Learning

Fengrui Xiao¹, Student Member, IEEE, Shuangwu Chen², Member, IEEE, Siyang Chen, Student Member, IEEE, Yuanyi Ma¹, Student Member, IEEE, Huasen He², Member, IEEE, and Jian Yang², Senior Member, IEEE

Abstract—Insider threats have become a prominent driver behind a myriad of cybersecurity incidents in recent years. Since the threats take place within intranet, traditional security devices located at the network perimeter can hardly detect them. The trust management methods employed within the organization are likewise incapable of intercepting access actions already authenticated with valid credentials. In this paper, we propose a novel insider threat detection method named SENTINEL, which identifies abnormal behavior of insiders and provides fine-grained threat intelligence. We devise a dynamic user behavior interaction graph (BIG), which jointly considers the spatial distribution of user behavioral trajectories among the network topology and the temporal variations of user behavioral profiles. By incorporating a spatio-temporal graph neural network, SENTINEL is able to learn the operation regularities of users at specific times and respective positions in BIG. In order to perceive both the abrupt and persistent threats simultaneously, we conceive a multi-timescale fusion mechanism for detecting users' activities at different timescales. SENTINEL implements a log-entry-level detection without requiring any attack samples during model training. The experiments conducted on widely-used public datasets demonstrate that SENTINEL achieves superior performance while maintaining a relatively low computational overhead compared to the state-of-the-art methods.

Index Terms—Insider threat detection, graph neural network, anomaly detection, behavior interaction graph (BIG).

I. INTRODUCTION

AMIDST the rapidly evolving landscape of network security, insider threats have become a prominent driver behind a myriad of cybersecurity incidents in recent years, which have inflicted substantial detriment upon government, enterprises and education networks. Insider threats commonly emerges from the actions of either malicious or unintentional insiders, who exploit their authorized access to negatively affect the reliability of the organization's information systems [1]. According to

Ponemon's 2022 Insider Threat Report [2], there has been a significant 44% increase in insider threat incidents over the past two years and the financial impact of each incident has risen by more than a third, with the amount of money lost per incident reaching a staggering \$15.38 million. In contrast to external attacks that need to breach security perimeter, attacks from insiders are more pernicious because the malicious insiders are well-acquainted with core secrets and may already have delegated authority to access sensitive data. Hence, insider threat poses serious security risks and potential harm to the organization's network.

Detecting insider threat still face many challenges for businesses and organizations of all sizes and across diverse industries. Firstly, due to their position within security perimeter, internal employees can inadvertently launch attacks without triggering alarms from security devices. Besides, these insiders with privileged access can leverage the credentials associated with their working accounts to circumvent the multi-layer trust management mechanisms within the organization [3]. Secondly, malicious activity carried out by insiders is relatively uncommon. As a result, there is often limited available data to describe and analyze such activities [4]. Thirdly, for the sake of exfiltrating confidential files, tampering with sensitive data, and sabotaging system components, insider threats perpetrators would move laterally across multiple hosts in the target network using stolen credentials and accounts [5]. The attacker's footprints could be fragmented and scattered across various user and host records, which makes it difficult to raise security personnel's attention. Lastly, well-planned attackers usually employ sophisticated strategies, like multistage persistent threats and mimicry attacks, to circumvent detection [6].

Massive efforts have been made to promptly detect insider threats. Existing insider threat detection methods can be roughly divided into three categories: rule-based methods, machine learning based methods, and graph-based methods [7]. Rule-based methods [8], [9], [10] employ manually defined rules to detect data leakage, which however heavily rely on expert knowledge, and thus can hardly handle attacks beyond predefined rules and prior experiences. Machine learning based [1], [4], [11], [12] approaches necessitate manual design of feature engineering and employ techniques such as Markov models or deep learning models to accomplish the detection task. Moreover, machine learning algorithms face the challenges of feature redundancy, feature correlation, and temporal-spatial feature

Received 5 November 2023; revised 24 October 2024; accepted 12 December 2024. Date of publication 17 December 2024; date of current version 24 February 2025. This work was supported in part by the National Natural Science Foundation of China under Grant U23A20275 and Grant 62101525 and in part by the China Environment for Network Innovations (CENI) under Grant 2016-000052-73-01-000515. Recommended for acceptance by Dr. Baochun Li. (Corresponding author: Shuangwu Chen.)

The authors are with the Department of Automation, University of Science and Technology of China, Hefei 230026, China (e-mail: chensw@ustc.edu.cn). Digital Object Identifier 10.1109/TNSE.2024.3519155

2327-4697 © 2024 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See <https://www.ieee.org/publications/rights/index.html> for more information.

processing [13] and rely on large volumes of labeled data to learn patterns of malicious activity. However, for insider threat detection, obtaining sufficiently high quality labeled data is very difficult. Lack of sufficient labeled data can result in inadequate or biased training of the model. Graph-based methods leverage the sequential relationships between log entries to learn normal patterns from graph structures (e.g. user authentication networks and lateral movement traces) [14], [15], [16], [17]. Existing graph-based insider threat detection methods usually treat operation logs as nodes and attempt to convert log records into a graph according to correlation relationships among them, which incurs high computational burden and lengthy execution times for large-scale graph learning.

Inspired by the limitations of current researches, we propose a novel insider threat detection method named SENTINEL, which provides a systematic and dynamic perspective on users' behavior within the intranet. Since insider threats often involve multiple stages, including reconnaissance, lateral movement, foothold establishment, and remote command and control, the behavior patterns of malicious insiders would exhibit temporal dependencies. Meanwhile, the network within organizations typically is comprised of multiple subnets and regions with specific network topologies. Thus, it is possible that the attackers need to traverse across various network regions or subnets to gain access to diverse systems during the execution of threat activities. This topological constraint indicates the spatial dependencies of insider threats. Based on this observation, we model user behaviors in different time periods into dynamic graph instances and develop an insider threat detection model using spatio-temporal graph neural network (ST-GNN). However, existing graph learning-based methods [14], [16], [18], [19] for insider threat detection generally treat logs as nodes, which results in extremely large graphs and significant computational overhead. In contrast, SENTINEL treats user interaction behavior logs between network hosts as edges rather than nodes of the graph, which greatly reduces the amount of nodes in graphs and thereby alleviates computational burden. Besides, analyzing insider threats requires investigating and processing diverse and voluminous users' log data [1], such as network traffic, file access records, logon authentication events, etc. Fusing such a substantial volume of data to capture complex relationships is a tricky problem. Nevertheless, traditional ST-GNN models only focus on the propagation of node attributes, but ignore the edges attributes. In this paper, edges represent user interactions between hosts, and their attributes often reflect the behavioral routines of different users. We can profile a user behavioral baseline based on the edge attributes, which plays a more critical role than the node attributes in our insider threat detection. To address this problem, we leverage an Edge Graph Attention (EGAT) mechanism [20] to acquire the feature representations of various operation types among system hosts, which allows us to capture the behavioral pattern implied by edge attributes. Moreover, in practical applications, abrupt threats and persistent threats may exist at the same time. To deal with a broad range of attack scenarios, we integrate user behavior states at multiple timescales into the detection model, which enables SENTINEL to capture dependencies in user behavior over different time

intervals [21]. Through this way, SENTINEL not only excels in rapidly responding sudden dangerous actions, but also can perceive the long-term evolutionary pattern of user behavior. Furthermore, to deal with the insufficiency of abnormal samples, a semi-supervised insider threat detection algorithm is proposed, which solely requires the training data to be benign.

Our contributions of this paper are fourfold as follows:

- We propose SENTINEL, an insider threat detection algorithm, to identify abnormal behavior of insiders and to provide fine-grained threat intelligence. SENTINEL implements a log-entry-level detection without requiring any attack samples during model training.
- We devise a dynamic user behavior interaction graph (BIG), which jointly considers the spatial distribution of user behavioral trajectories among the network topology and the temporal variations of user behavioral profiles. By incorporating ST-GNN with edge feature enhancement, the BIG achieves accurate modeling of spatio-temporal relationships between nodes, enhancing the representative ability of user behaviors.
- In order to perceive both the abrupt and persistent threats simultaneously, we conceive a multi-timescale fusion mechanism for incorporating users' activities at different timescales. With the help of the temporal hierarchy aggregation layer, the multi-level temporal interaction context is explicitly embedded into the potential representation of timescale-aware.
- We conduct experiments on three public datasets to compare the performance of SENTINEL to the state-of-the-art methods. The results demonstrate that SENTINEL achieves superior AUC value of 0.980 while maintaining a relatively low computational overhead.

The rest of our paper is organized as follows. Section II summarizes the related work. Section III describes our threat model and challenges, and the detailed system architecture is proposed in Section IV. Section V presents the comparative experiments. Finally, we conclude this paper in Section VII.

II. RELATED WORK

Existing Researches on insider threat detection can be divided into three categories: rule matching based methods, machine learning based methods, and graph-based methods.

Rule matching has been widely applied in insider threat detection. Early detection tools, such as Addamark LMS [8] and Splunk [9], primarily generated alerts by monitoring users' log files or network requests and matching configurable rule lists. With the help of these tools, security analyst could determine the pertinent log files to assess and prescribe appropriate alert actions [22]. Although these offline tools or schemes could be executed multiple times within an hour or a day, they lacked the capacity to conduct longitudinal log analysis or proactively respond in real-time. To remedy these deficiencies and deal with new emerged Advanced Persistent Threat (APT), Milajerdi et al. [10] utilized the Tactics, Techniques, and Procedures (TTPs) abstract rules based on MITER's ATT&CK framework [23], and mapped the low-level audit logs to the high-level

attack kill chain stages. Though rule matching approaches have demonstrated favorable performance in insider threat scenarios, they necessitate substantial expert and prior knowledge [16] and face challenges in adapting to the constantly evolving landscape of network attacks.

Given massive data generated daily within organizations, machine learning based detection methods have emerged as one of the most promising ways for detecting insider threats. Senator et al. [24] employed machine learning algorithms such as Hidden Markov Models (HMM) and Gaussian Mixture Models (GMM) to detect subtle threat signals within enterprise data systems. However, these static features often cannot reflect the dynamic changes of user behavior. Recognizing that user behavior evolves over time, Chattopadhyay et al. [12] proposed a technique to detect insider threat scenarios by constructing time series with the statistics features of each single day feature. Due to the variable duration of threat activities, Le et al. [4] devised a user-centric insider threat detection system with multiple levels of data granularity, employing several classic machine learning algorithms. Meng et al. [25] introduced a trust management approach in healthcare environments based on Bayesian inference to identify malicious devices. Le et al. [1] investigated the effectiveness of various anomaly detection algorithms in the context of insider threat scenarios. In general, traditional shallow machine learning algorithms achieve commendable accuracy in insider threats detection. Nevertheless, these methods often rely on manual feature engineering, which significantly restricts their flexibility.

By contrast, deep learning, renowned for its ability to learn multi-level hidden representations from intricate data, can serve as a powerful tool to identify potential malicious insider activity. In [26], Tuor et al. proposed an adaptive detection algorithm based on Recurrent Neural Network (RNN) language models. They trained an online unsupervised neural network language model by treating system logs as interleaved “sentences”. In order to further predict the specific steps the adversary would take when performing the attack, Shen et al. from Symantec Labs proposed Tiresias [11], an Long Short Term Memory (LSTM) based system designed for predicting future attack events. Randive et al. [27] introduced an innovative image-based insider threat detection framework that utilizes wavelet transform and capsule networks to discern the spatial arrangement and relationships among internal user behavior features. Deep learning methods possess the capability to capture temporal information from user activity sequences, but they would encounter high false alarm rates when users modify their daily behavioral routines.

Considering that users within an organization conduct their daily activities through email, shared files or public devices, utilizing a graph structure to illustrate the relationships and dependencies between user behaviors is a reasonable approach. In [28], Eberle et al. utilized the Minimum Description Length (MDL) to learn normative patterns from the entire graph that capture the correspondence of insiders. Similarly, Bian et al. [15] constructed authentication graphs based on login authentication events between hosts. Traditional machine learning techniques are then applied to detect hosts in the network that may be targeted by insiders. Inspired by graph embedding techniques, Liu et al. [14] utilized heuristic rules to build a heterogeneous graph

where log entries represented nodes. Afterwards, they employed Deepwalk [29] to generate node embedding vectors, which were subsequently input into clustering algorithms for anomaly detection. To identify and track lateral movement of insiders within the intranet, Fang et al. [19] proposed LMTracker, an attack path detection algorithm based on meta-path and heterogeneous graph learning. It should be noted that when the data cannot be readily represented as a graph structure, the construction of the graph becomes a laborious task, often requiring extensive manual engineering efforts [14].

In recent years, Graph Neural Networks (GNNs) have gained popularity in graph analysis and have been successfully applied to anomaly detection in organizational information networks [30]. Li et al. [5] designed a hierarchical detection method. They introduced a meta-path aggregation GNN and an edge-augmented GNN for embedding provenance graphs and host interaction graphs respectively. Xiao et al. [16] proposed a multi-edge weight relationship graph neural network (MEWRGNN) for robust anomaly detection, which converted time series related to user behavior logs into graphs. Li et al. [17] proposed a modular approach called Dual-domain Graph Convolutional Network (DD-GCN), which transformed user features and structural data into heterogeneous graphs to achieve accurate and adaptive insider threat detection.

Currently, few studies take into account the dynamic migration of user activities across the network topology to identify the criminals’ activity path and behavioral profiles in multi-host intranet environment. Actually, there are significant differences in the temporal and spatial distribution of attack activities in the insider threat scenarios, which lead to potential false alarms and intelligence omissions. In light of this, we conceive a novel insider threat detection algorithm based on multi-timescale ST-GNN. We compare the key characteristics of the related studies and our current work on detecting insider threats in Table I.

III. THREAT MODEL AND MOTIVATION

A. Threat Model

Insider threats commonly arise from internal employees or hackers who infiltrate internal networks through APTs. These cybercrime initiators usually possess authorized access to systems and deliberately exploit it to jeopardize the confidentiality, integrity, or availability of those systems [7]. In practice, insider threats typically involve multi-step attacks, wherein attackers orchestrate a series of individual steps based on specific logical relationships.

Particularly, we consider an insider threat conducted by an institution employee who is familiar with business logic and does not initially have administrative privileges assigned to a local computer. This employee may hold a technical position such as a systems analyst or IT support personnel, which grants him/her access to partial sensitive information. Such attackers may have different expertise but generally possess knowledge of computer network protocols and architecture, as well as experience with scripting/programming languages such as Python, PowerShell, and Bash. Additionally, they are likely to be well-versed in the company’s security policies. These attackers may be dissatisfied with their current situation in the company (e.g. frustrated by

TABLE I
INSIDER THREAT DETECTION METHODS WITH CORRELATION ANALYSIS

Reference	Methodology	Detection Granularity	Spatial Relation	Temporal Relation	Multi-Timescale	Malicious Training Samples Independence	Expert Knowledge Independence
[8], [9]	Rule Matching	Log Entry	×	×	×	×	×
[10]	Attack Model	Log Entry	×	×	×	×	×
[24]	HMM	Log Entry	×	✓	×	×	✓
[25]	Trust Management	Network Endpoint	✓	✓	×	×	✓
[4], [12], [15]	MLP&RF&XGB&LR	User&User Session	×	✓	✓	×	✓
[11]	Anomaly Detection	User&User Session	×	✓	✓	✓	✓
[6], [11], [26]	LSTM	Log Entry	×	✓	×	✓	✓
[28]	MDL	Log Entry	✓	×	×	✓	✓
[14], [31]	Graph Learning&Clustering	Log Entry	✓	✓	×	✓	✓
[16], [17]	GNN	User&User Session	✓	×	×	×	✓
[5]	GNN	Log Entry	✓	✓	×	✓	✓
[19]	Graph Learning	Log Entry	✓	×	×	✓	✓
[27]	Capsule Network	User Session	✓	×	×	×	✓
Proposed	ST-GNN	Log Entry	✓	✓	✓	✓	✓

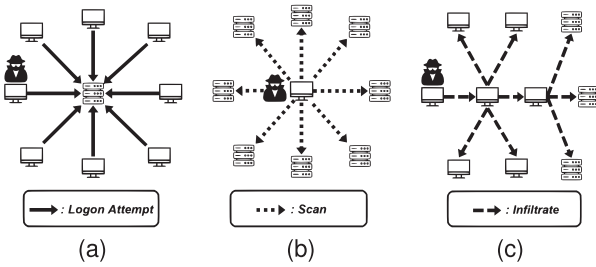


Fig. 1. Examples of three abnormal activities of insider threat with different spatial characteristics: (a) In-star. (b) Out-star. (c) K-path.

lack of promotions or financial incentives) and deliberately abuse their access for malicious revenge, economic gain, or both. Consequently, they tend to target valuable and confidential information, like clients' details and tax records stored on the database server.

With the aim of surreptitiously exfiltrating confidential information, the attackers usually execute the following steps: 1) escalate their user privileges through illegal levers such as credentials theft; 2) move between machines via login or remote command execution events with stolen credentials for authentication; 3) launch covert and persistent attacks towards target accounts and hosts to acquire confidential data; 4) exploit already installed applications such as Powershell, and run malicious scripts to establish remote connections for transferring stolen data. Traditional enterprise security solutions, such as firewalls, intrusion detection systems (IDS), and intrusion prevention systems (IPS), are primarily implemented at network perimeter, and thus cannot adequately detect and prevent insider threats.

B. Motivation

In real-world system, attackers' historical operation data exhibit distinct spatio-temporal patterns and behavioral profiles that deviate from normal service logic, and such characteristics are widely distributed in data with multiple timescales.

Spatio-temporal Patterns. Typical insider threats may involve multiple stages such as reconnaissance, lateral movement, and foothold establishment, which will give rise to serial event sequences in the log system. These sequential information not only possess temporal dependencies but also often have the characteristics of spatial patterns. Fig. 1 illustrates the communication characteristics that an attacker may exhibit during each stage: a)

In-Star. Multiple accounts attempt to log into a critical server. b) Out-Star. Attackers systematically scan a range of internal network nodes in search of vulnerable hosts with open services and exploitable vulnerabilities. c) K-Path. Attackers initiate their attack from a foothold on a host, and systematically migrate and infiltrate the internal network, seeking out target hosts to store critical data. All of these activity patterns show distinct spatial topological patterns, which can be easily obscured by the abundance of parallel irrelevant data in serialized data. However, attackers' lateral movement through compromised accounts has a common feature that the access path of attackers does not include systems/hosts frequently accessed by compromised accounts [32]. This unusual access/behavior trajectory can be a precursor to stolen credentials or dangerous operations. Besides, crucial hosts with masses of critical data, e.g. database servers, are often selected as targets of cybercrime. The interaction between users and nodes in such networks contains more effective information. By analyzing the spatio-temporal patterns of users, the detection system can learn the operation regularities of users at specific times and respective positions. This is beneficial to detecting any activities that deviate from the established normal patterns.

Behavioral Profiles: Behavioral profiles, which include relevant information such as logon type, authentication type, logon timestamp, etc., can describe and analyze the behavioral characteristics of an individual. By constructing users' behavior profiles, we can gain an insight into their normal operation routines and attributes on each host. In some cases, they can even reveal some potential threat behaviors. The attribute changes of user operations in the same hosts may indicate the presence of potential security risks. For example, the attacker can exploit stolen accounts to remotely access a database server, and then implant a backdoor for the purpose of monitoring the flow of information and stealing credentials. If the Service Principal Name (SPN) of this server is registered in the Active Directory, the victim would have accessed the server through the Kerberos protocol to retrieve information. However, other services (e.g. backdoor software) are not registered in Active Directory. At this point, the attacker has to replace Kerberos with another protocol, NTLM. In this case, changes in the behavioral profile should be of concern. Behavior profiles offers a comprehensive perspective for detecting and identifying potential security risks and threat behaviors.

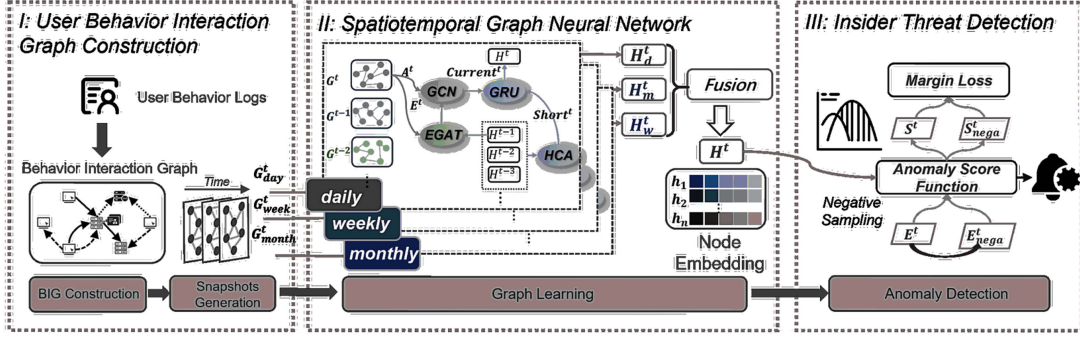


Fig. 2. The framework of SENTINEL, which mainly consists of three stages: 1) BIG construction. 2) ST-GNN based behavioral dependency learning. 3) Insider threat detection.

Multiple Timescales: Due to the high uncertainty of insider threat, it is necessary to fuse multiple timescales in the detection scheme to describe the behavioral patterns of users. On one hand, insider threats occur at different frequencies, some events may occur with low frequency but high risk, while others may occur with high frequency but low risk. By building a multi-timescale model, it becomes possible to analyze and compare low-frequency and high-frequency events to normal behavior patterns simultaneously. On the other hand, users' behavior baselines may change over time. Persistent changes over time may indicate a potential insider threat, but infrequent behavioral patterns in the short term could still be normal or accidental. Fusing the multi-timescale context information can contribute to handling alerts of suspicious behavior and clarifying the boundary between malicious behavior and normal operation.

IV. INSIDER THREAT DETECTION BASED ON EDGE FEATURE ENHANCED SPATIO-TEMPORAL GRAPH NEURAL NETWORK

A. Overview

We propose a novel framework for detecting insider threats, named "SENTINEL", as shown in Fig. 2. SENTINEL infers potential insider threats that may endanger the system security from large amounts of heterogeneous users' behavior logs collected by geographically dispersed servers and terminals in the internal network.

SENTINEL consists of three main parts:

1) **User Behavior Interaction Graph Construction:** The existing computer system is essentially a distributed system comprised of multiple hosts, network components, and storage devices. Sensitive information may be stored in the storage devices or across multiple nodes within this system. This graph provides us a holistic view of structural interactive topology and a detailed users' behavioral profile. Under this situation, users' requests may be transmitted and processed among different hosts to fetch their desired resources. The interactive behaviors across internal systems are naturally formed as a graph, named Behavior Interaction Graph (BIG), where the internal network hosts and the users' operational behaviors across hosts denote nodes and edges, respectively.

2) **Spatio-Temporal Graph Neural Network Model:** We employ ST-GNN, which integrates GNN and temporal model,

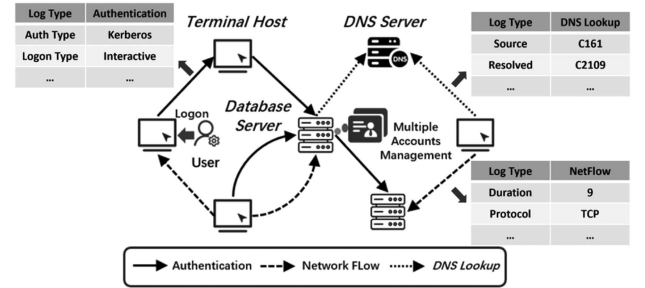


Fig. 3. An example of user behavior interaction graph where nodes represent diverse network hosts (such as DNS server, database, users' terminal, etc.) and edges represent different interaction events (such as authentication, network transfers and DNS lookup, etc.) with multidimensional information.

to learn the spatio-temporal dependencies of user behavior. Considering the uncertainty of insider threats, SENTINEL simultaneously extracts state information at multiple timescales and proposes an aggregation layer to model the underlying dependencies of multiple temporal dynamics. This modeling scheme is beneficial to perceiving propagation paths of insider threats, and promptly identifying potential sources of threats.

3) **Insider Threat Detection:** We assess the threat degree of behavioral events by utilizing the learned node feature representations. To adapt to the extremely sparse abnormal data in network security logs, we use negative sampling to train the entire network and implement a semi-supervised training model.

B. User Behavior Interaction Graph Construction

We adopt user behavior interaction graphs to model users' interaction in the intranet. Fig. 3 presents an example of BIG where users log into their work accounts, and produce various types of operational events which are represented by different types of edges. Since individuals within an organization have diverse tasks and positions, the edges exhibit multiple attributes in the graph. For example, administrators usually access different hosts for network configuration, database maintenance and risk assessment, whereas the common users frequently perform file transmissions and Internet access, which may generate vast quantities of network stream records and DNS traffic logs. To depict these inter-host behaviors, BIG needs to consider the

effects of multiple types of events simultaneously. Formally, a BIG is denoted by $\mathbf{G} = (\mathbf{V}, \mathbf{E})$, where $\mathbf{V}$ is the set of vertices, each $v \in \mathbf{V}$ representing a host node or user entity in the local network topology; $\mathbf{E}$ is the set of edges, each $e \in \mathbf{E}$ representing an event that occurs between different hosts. Let $\mathbf{H} = [\mathbf{h}_1, \mathbf{h}_2, \dots, \mathbf{h}_N] \in \mathbb{R}^{N \times d}$ denote the node embedding in the graph where N represents the number of graph nodes and d is the dimension of node embedding. The connection relationship between nodes in the graph can be represented by an adjacent matrix $\mathbf{A}$, where $\forall e = (i, j) \in \mathbf{E}$, $\mathbf{A}[i][j] = 1$.

To capture temporal dependencies in user behaviors, SENTINEL dissects the long-term log stream into a sequence of short-term snapshots, each of which encapsulates all interactive events taking place within a discrete time interval (e.g., hours). Let T denote the total number of time snapshots. A BIG stream is denoted by $\{\mathbf{G}^t\}_{t=1}^T$, where each $\mathbf{G}^t = \{\mathbf{V}^t, \mathbf{E}^t\}$ represents the BIG snapshot at time t . For convenience, let $\mathcal{V} = \cup_{t=1}^T \mathbf{V}^t$ and $\mathcal{E} = \cup_{t=1}^T \mathbf{E}^t$ denote the set of nodes and edges across all time intervals, respectively. Given the union of BIG stream $\{\mathbf{G}^t\}_{t=1}^T = (\mathcal{V}, \mathcal{E})$, the goal of SENTINEL is to detect malicious events (i.e. edges in the graph $\mathcal{E}$) caused by insider threats.

C. Edge Feature Enhanced Graph Embedding

BIG describes a series of communication and interaction actions of users within intranet. These actions are characterized as multi-attribute edges in the BIG, which can help us understand the behavioral habits of users in their daily work and identify abnormal operation sequences. Hence, the edge of BIG plays a more important role than node in insider threat detection, that is, BIG is *edge-sensitive*. However, the existing graph learning methods are typically node-centric, which fail to effectively leverage edge features. Based on such node-centric graph, previous graph-based insider threat detection methods [14], [16], [33] treat operation logs as nodes and attempt to convert log records into a graph according to correlation relationships among them. However, since the number of user logs may be several orders of magnitude higher than the number of network endpoints, the size of the graph is very huge, which makes these methods encounter significant computational complexity issues. Hence, in this section, we propose a graph embedding method to aggregate node features with edge attributes for enhancing the model's ability to capture complex relationships between nodes.

The raw operation logs consist of both numerical and textual fields. For the numerical fields, such as network flow bytes and port numbers, are already represented as numerical values and do not require any conversion. In order to handle the unstructured text in the raw operation logs, we have to transform them into a numerical vector so that the learning-based detection model can handle the logs. The operation logs typically contain two kinds of textual fields: 1) *discrete text* often represents the distinct operation objects and types, such as logon authentication protocol and logon type, which is predefined and thus has a finite value range. We can employ simple ordinal encoding to convert the *discrete text* into a numerical vector. 2) *continuous text* typically represents the description of accessed resources, such as file paths and file contents, which is comprised of

symbols, words, or sentences. Following the approach in [16], we utilize the Word2Vec [34] to convert the *continuous text* into a numerical vector. In fact, different operations may possess distinct log formats (i.e. containing different fields), which may lead to a potential escalation in the logical intricacies of data processing. To address this problem, we create a uniform log format containing all the fields appeared in the logs. If a certain log record does not contain a specific field, we can pad such field with 0. Suppose the operation logs contain K distinct fields and each field generates a corresponding numerical vector $\mathbf{f}^k$. Then, we can get a feature vector for each log by cascading all numerical vectors, i.e. $\mathbf{F}_{ij} = \mathbf{f}_{ij}^1 || \mathbf{f}_{ij}^2 || \dots || \mathbf{f}_{ij}^K$. If a user has an interactive event between nodes i and j , there exists an edge between them, which is represented by $(i, j, \mathbf{F}_{ij})$. Since users may interact with other hosts after logging into a source host, we use an attention mechanism to aggregate these operational features. Actually, different log field exhibits different importance for threat detection. For example, the usage of removable devices are likely indicative of data breach. Hence, given an edge $e = (i, j, \mathbf{F}_{ij})$ in $\mathbf{G} = (\mathbf{V}, \mathbf{E})$, we regard the K fields of $\mathbf{F}_{ij}$ as K -channel signals, and utilize a separate attention network for each channel to update the original node features $\mathbf{H}$. For the k -th field, the attention coefficient α_{ij}^k is calculated as follows:

$$\beta_{ij}^k = a(\mathbf{W}_h^k \mathbf{h}_i || \mathbf{W}_h^k \mathbf{h}_j || \mathbf{W}_e^k \mathbf{f}_{ij}^k), \quad (1)$$

$$\alpha_{ij}^k = \frac{\exp(\text{LeakyReLU}(\beta_{ij}^k))}{\sum_{s \in N_i} \exp(\text{LeakyReLU}(\beta_{is}^k))}, \quad (2)$$

where $\mathbf{W}_h^k$ and $\mathbf{W}_e^k$ are the weight matrices for the k -th channel, $\mathbf{h}_i$ and $\mathbf{h}_j$ are the node feature vectors of node i and node j respectively, N_i is the neighboring of node i , and $a(\cdot)$ is a linear mapping function.

By concatenating the outputs of each channel, we can update the node embedding vector as follows:

$$\mathbf{h}_i' = (||_{k=1}^K \sigma(\sum_{j \in N_i} \alpha_{ij}^k (\mathbf{W}_h^k \mathbf{h}_i || \mathbf{W}_e^k \mathbf{f}_{ij}^k))) \mathbf{W}_o, \quad (3)$$

where $\mathbf{W}_o$ is a projection weight vector and $\sigma(\cdot)$ is the Sigmoid function. In this way, the whole graph is embedded as follows:

$$\mathbf{H}' = \text{EGAT}(\mathbf{H}, \mathbf{E}), \quad (4)$$

where $\mathbf{H}'$ is the updated embedding vector of all nodes in $\mathcal{V}$. The above aggregation process is illustrated in Fig. 4.

Aggregating the edge features can enhance the ability of BIG to represent the behavioral routines of insiders. Since the embedding of a node fuses the features of all the edges originating from such node, the node embedding characterizes all interactive operations launched by this node (i.e. host or user) towards other system entities. Therefore, we can employ the widely-used node-centric graph learning method to handle the *edge-sensitive* BIG. As the size of BIG depends on the number of hosts in the intranet which is far less than the number of logs, the computational burden of graph learning can be greatly alleviated.

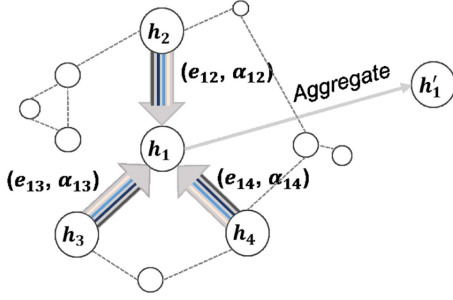


Fig. 4. Illustration of edge graph attention layer. Each edge is composed of K channels, denoted by different colors. $\mathbf{F}_{ij}$ is the feature vector of the directed edge e_{ij} and α_{ij} is the attention factor calculated through (2). $\mathbf{h}_i$ indicates the initial node embedding of node i and $\mathbf{h}'_1$ is the updated node embedding which aggregates edge information through (3).

D. Spatio-Temporal Behavioral Dependencies Learning

The insider threat activities typical exhibit obvious spatio-temporal pattern as illustrated in Section III-B. In light of this, we attempt to learn the operation regularities of users at specific times and positions from the above edge-enhanced BIG. As ST-GNNs can effectively capture both the spatial and temporal characteristics of graphs, we adopt a GCN+GRU model [35] to learn the spatio-temporal behavioral dependencies from BIG.

At time t , the snapshot of user behavioral logs is modeled as a BIG $\mathbf{G}^t = \{\mathbf{V}^t, \mathbf{E}^t\}$ with an adjacent matrix $\mathbf{A}^t$ and a node embedding matrix $\mathbf{H}^t \in \mathbb{R}^{N \times d}$. Here, an L -layer GCN [36] is exploited to capture the spatial correlations between nodes in BIG. The spatial embedding at l -th layer is described as

$$\mathbf{Spat}^{(l)} = \text{ReLU}(\tilde{\mathbf{D}}^{-\frac{1}{2}} \tilde{\mathbf{A}}^t \tilde{\mathbf{D}}^{-\frac{1}{2}} \mathbf{Spat}^{(l-1)} \mathbf{W}^{(l-1)}), \quad (5)$$

where $\mathbf{Spat}^{(0)} = \mathbf{H}^t$, $\tilde{\mathbf{A}}^t = \mathbf{A}^t + \mathbf{I}_N$ is the regularized adjacent matrix with self loops, and $\tilde{D}_{ii} = \sum_{j \neq i}^N A_{ij}^t$ denotes the degree of node i . The propagation process of GCN describes how intensively hosts or users in the internal network interact with other system entities. $\mathbf{Spat}^{(L)}$ represents the structural patterns of users' behaviors.

The enormous logs generated by users' daily activities within the internal network may form a large number of graph snapshots. Analyzing the association among all the snapshots may lead to *dependency explosion* [37]. Therefore, we use a sliding time window with a size of ω to capture the temporal dependencies among graph snapshots over a short period of time. At time t , a historical sequence of embedding of node i over recent snapshots is denoted by $\mathbf{C}_{h,i}^t = [\mathbf{h}_i^{t-\omega}, \dots, \mathbf{h}_i^{t-1}]$. We use a hierarchical contextual attention (HCA) based model [38] to capture the sequential dependencies of $\mathbf{C}_{h,i}^t$ as follows:

$$\mathbf{a}_{h,i}^t = \text{softmax}(\mathbf{r}^T \tanh(\mathbf{Q}_h(\mathbf{C}_{h,i}^t)^T)), \quad (6)$$

where $\mathbf{Q}_h$ and $\mathbf{r}$ are weight parameters. $\mathbf{a}_{h,i}^t$ measures the impact of the historical behaviors of node i on its current behavior. The temporal embedding of all nodes in $\mathcal{V}$ is described as:

$$\mathbf{Temp}^t = [\mathbf{temp}_1^t, \dots, \mathbf{temp}_i^t, \dots, \mathbf{temp}_N^t], \quad (7)$$

where $\mathbf{temp}_i^t = (\mathbf{a}_{h,i}^t \mathbf{C}_{h,i}^t)^T$.

In order to combine the spatial and temporal dependencies, we employ a GRU to fuse $\mathbf{Spat}^{(L)}$ with $\mathbf{Temp}^t$, where $\mathbf{Spat}^{(L)}$ and $\mathbf{Temp}^t$ serve as the current input and the hidden state of GRU, respectively. The GRU is composed of an update gate $\mathbf{Z}^t$ and a reset gate $\mathbf{R}^t$, which are calculated by:

$$\mathbf{Z}^t = \sigma(\mathbf{U}_Z \mathbf{Spat}^{(L)} + \mathbf{W}_Z \mathbf{Temp}^t + \mathbf{b}_Z), \quad (8)$$

$$\mathbf{R}^t = \sigma(\mathbf{U}_R \mathbf{Spat}^{(L)} + \mathbf{W}_R \mathbf{Temp}^t + \mathbf{b}_R), \quad (9)$$

where $\sigma(x) = 1/(1 + e^{-x})$ is the Sigmoid function, and $\mathbf{W}_Z, \mathbf{W}_R, \mathbf{U}_Z, \mathbf{U}_R, \mathbf{b}_Z, \mathbf{b}_R$ are weight matrices and biases matrices of GRU. Reset gate $\mathbf{R}^t$ determines the relevant past information to incorporate when generating a new state $\tilde{\mathbf{H}}^t$:

$$\tilde{\mathbf{H}}^t = \tanh(\mathbf{U}_h \mathbf{Spat}^{(L)} + \mathbf{W}_h (\mathbf{R}^t \odot \mathbf{Temp}^t)), \quad (10)$$

where $\odot$ is the element-wise multiplication operator, and $\mathbf{W}_h, \mathbf{U}_h$ are weight matrices. Afterwards, the update gate $\mathbf{Z}^t$ removes the forgotten information from the previous states, and retains the information that need to be remembered. Finally, we can get a graph embedding $\mathbf{H}^t$ fusing both spatial and temporal dependencies as follows:

$$\mathbf{H}^t = (1 - \mathbf{Z}^t) \odot \mathbf{Temp}^t + \mathbf{Z}^t \odot \tilde{\mathbf{H}}^t. \quad (11)$$

The final graph embedding $\mathbf{H}^t$ not only captures the temporal dependencies of user behavior, but also aggregates the structural features of users' interactive trails.

E. Multiple Timescales Fusion

Insider threat activity may emerge abruptly in a brief time (e.g a half day) or persist gradually within the system over a long duration (e.g several months). Hence, monitoring the alterations of user behavior solely in a single timescale is insufficient to perceive these uncertain insider threats. A short-timescale monitoring is beneficial for the detection of abrupt abnormal behaviors, although it may introduce more noise and false positives. Whereas, a long-timescale monitoring contributes to capturing the persistent abnormal behavior patterns, which may overlook subtle but crucial events. The insider threat detection system should monitor user behaviors at different timescales. To achieve this goal, current researches [1], [4], [12] train a separate detection model for each timescale, which incurs extra storage and computation burden, and more importantly, may increase false alarms. The regular actions that are conducted infrequently in a short term are easily misidentified as attacks, for example the data backup by administrators could be regarded as data breaches. Therefore, we attempt to fuse multi-timescale BIG embedding in this section.

When fusing the embedding of different timescale, the embedding of different node at different timescales has a distinct weight. Taking the CERT dataset [39] as an example, the interactive behaviors between some hosts have obvious peak hours in the morning and evening. In this situation, the daily-period and weekly-period detection are more critical than monthly-period. However, there are no distinct behavioral patterns for some other hosts, thus the daily-period and weekly-period detection may be unhelpful. Here, we use a gate mechanism to learn the weights of node embedding at different timescales.

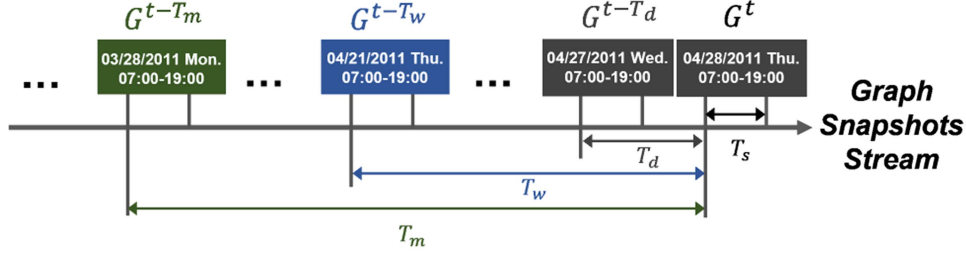


Fig. 5. An example of fetching snapshot into the sliding windows at different timescales. T_s represents the duration of a snapshot which is half day in the figure. T_d , T_w and T_m denote the time interval of daily-period, weekly-period, and monthly-period respectively. G^t represents a graph snapshot fetched at time t .

We analyze the BIG embedding at three timescales, i.e. daily-period, weekly-period, and monthly-period, the time intervals of which are denoted by T_d , T_w , and T_m , respectively. As mentioned in Section IV-D, we capture the temporal dependencies among graph snapshot in the sliding window ω . Here, as illustrated in Fig. 5, we fetch the snapshots in the same time of past few days, weeks or months and put them into the daily-period, weekly-period, and monthly-period sliding windows, respectively. The snapshots in the three-timescale sliding windows are defined as:

$$\mathbf{G}_p^t = [G^{t-(\omega-1)*T_p}, \dots, G^{t-T_p}, G^t] \quad (12)$$

where $p \in [day, week, month]$ represent the timescale.

Through aforementioned spatio-temporal dependency learning, we can acquire the node embedding $\mathbf{H}_d^t$, $\mathbf{H}_w^t$, $\mathbf{H}_m^t$ at different timescales. The multi-timescale node embedding is fused as follows:

$$\mathbf{H}^t = \mathbf{W}_d \odot \mathbf{H}_d^t + \mathbf{W}_w \odot \mathbf{H}_w^t + \mathbf{W}_m \odot \mathbf{H}_m^t. \quad (13)$$

where $\odot$ is the element-wise multiplication operator, and $\mathbf{W}_d$, $\mathbf{W}_w$, $\mathbf{W}_m$ are linear parameters for different timescales. $\mathbf{H}^t$ is the weighted sum of multi-timescale node embedding.

F. Negative Sampling and Anomaly Detection

Due to the diversity and stealthiness of insider threats, it is almost impossible to acquire adequate labeled data for comprehensively describing their attack tactics. Additionally, there is serious data imbalance where the quantity of normal behavior samples far exceeds abnormal behavior samples. Therefore, conventional supervised methods are not suitable for the scenario of insider threat detection. To address this issue, we train the detection model using negative sampling, which eliminates the requirement for balanced and well-annotated samples. Afterwards, we propose anomaly scores to quantify the degree that a behavior deviates from the baseline of normal behavior, so that the anomalous edges can be detected in a semi-supervised way.

To surmount the insufficiency of anomaly data, we adopt the negative sampling [40] to generate an anomalous edge for each normal edge. Intuitively, abnormal edges are generated when insiders establish interactions with the hosts that are rarely accessed. For a graph snapshot $\mathbf{G}_t = (\mathbf{V}_t, \mathbf{E}_t)$, we generate $|\mathbf{E}_t|$ new edges as the candidates of negative edges by randomly replacing the endpoints of normal edges with other nodes in

the graph. Specifically, we define a Bernoulli distribution for negative sampling. Given a normal edge $e = (i, j, \mathbf{F}_{ij})$, we replace the source host i with the probability of $\frac{d_i}{d_i + d_j}$ or the destination host with the probability of $\frac{d_j}{d_i + d_j}$, where d_i and d_j represent the degrees of node i and node j , respectively.

Even though we have corrupted normal samples with negative sampling, there still exists the possibility that the generated edges turn out to be normal. Hence, we use a critic network to evaluate the anomaly score as follows:

$$S(e) = \sigma(\mathbf{w}_s(\mathbf{h}_i || \mathbf{h}_j || \mathbf{F}_{ij}) + b_s), \quad (14)$$

where $\mathbf{h}_i$ and $\mathbf{h}_j$ denote the embedding of node i and node j respectively, $\sigma(\cdot)$ is the Sigmoid function, and $\mathbf{w}_s$ and b_s are network parameters. Moreover, strict loss functions such as cross-entropy is not feasible to distinguish original edges from generated edges. Instead, we employ an edge-based pairwise loss function in the training of SENTINEL:

$$\mathcal{L}^t = \min \sum_{e \in \mathbf{E}^t} \max\{0, \gamma + S(e) - S(e_{neg})\}, \quad (15)$$

where $S(\cdot)$ is the anomaly score function, e_{neg} is the generated negative edge, $\gamma \in (0, 1)$ is the margin (hyperparameter) between the possibilities of normal edge and anomalous edge, and $S(e)$ and $S(e_{neg})$ are the anomaly scores of original samples and negative samples, respectively.

The minimization of the loss $\mathcal{L}^t$ encourages that the anomaly score of normal samples become smaller while the anomaly score of generated negative samples become larger, which fits with our aim perfectly. In addition, we introduce L2 regularization loss to avoid model overfitting. The final loss function is summarized as follows:

$$\mathcal{L} = \mathcal{L}^t + \lambda \mathcal{L}_{reg}, \quad (16)$$

where λ is a hyperparameter and $\mathcal{L}_{reg}$ is an L2-regularization loss.

We provide the training process of SENTINEL in Algorithm 1. SENTINEL trains in an iterative and end-to-end manner. In each iteration, each edge is first undergoing negative sampling to generate a negative edge (line 5). Then, for all positive and negative edges, we conduct edge feature enhancing, spatio-temporal dependency learning, multi-timescale feature fusing, and anomaly score computing successively (lines 6-11).

Algorithm 1: The Training Process of SENTINEL.

Input: User behavior logs x collected from the internal network, Number of training epochs I .
Output: Anomaly score $S(\cdot)$ of logs x .

- 1: Parse log x into graph edges $\{\mathbf{E}^t\}_{t=1}^T$, and generate graph snapshots sequences $\{G^t\}_{t=1}^T$.
- 2: **for** $i = 1$ to I **do**
- 3: **for** $t = 1$ to T **do**
- 4: **for** e in $\mathbf{E}^t$ **do**
- 5: $e_{neg} \leftarrow$ negative sampling;
- 6: $\mathbf{H}' \leftarrow$
 $\mathbf{h}'_i = (\|_{k=1}^K \sigma(\sum_{j \in N_i} \alpha_{ij}^k (\mathbf{W}_h^k \mathbf{h}_j \| \mathbf{W}_e^k \mathbf{f}_{ij}^k))) \mathbf{W}_o$
for each node i ;
- 7: $\mathbf{Spat}^{(L)} \leftarrow \mathbf{Spat}^{(l)} = GCN(\mathbf{Spat}^{(l-1)})$ for l
in range (L) where $\mathbf{Spat}^{(0)} = \mathbf{H}'$;
- 8: **for** p in timescales {day, week, month} **do**
- 9: $\mathbf{Temp}^t \leftarrow \mathbf{temp}_i^t = (\mathbf{a}_{h,i}^t, \mathbf{C}_{h,i}^t)^T$ for each
node i ;
- 10: $\mathbf{H}_p^t = GRU(\mathbf{Spat}^{(L)}, \mathbf{Temp}^t)$;
- 11: **end for**
- 12: $\mathbf{H}^t \leftarrow \mathbf{W}_d \odot \mathbf{H}_d^t + \mathbf{W}_w \odot \mathbf{H}_w^t + \mathbf{W}_m \odot \mathbf{H}_m^t$
- 13: $S(e), S(e_{neg}) \leftarrow S(\cdot) =$
 $\sigma(\mathbf{w}_s(\mathbf{h}_i \| \mathbf{h}_j \| \mathbf{F}_{ij}) + b_s)$
- 14: **end for**
- 15: $\mathcal{L}^t \leftarrow \min_{e \in \mathbf{E}^t} \max\{0, \gamma + S(e) - S(e_{neg})\}$
- 16: Back propagation and update the parameters;
- 17: **end for**
- 18: **end for**

Finally, the parameters are updated through back propagation under the supervision of loss function defined in (15).

During the detection phase, we can acquire an anomaly score for each edge in BIG, which measures the degree that a behavior deviates from the baseline of normal behavior. Hence, we set a threshold as the anomalous boundary to detect which behavior is abnormal.

V. EXPERIMENT

In this section, we present the evaluation of our proposed SENTINEL. Specifically, we first introduce the datasets. Then, the experimental setup are elaborated, including the comparative baselines and the metrics for evaluation. Finally, the effectiveness and accuracy of SENTINEL are evaluated, and the comparison experiments with the baseline methods are also presented.

A. Dataset

We evaluate the performance of our proposed SENTINEL on three publicly available datasets including LANL dataset, CERT and TWOS dataset, which are widely employed in the field of insider threats detection [1], [4], [5], [12], [14], [15], [16], [31]. The detailed description are demonstrated as follows.

LANL Dataset: LANL dataset is generated by Los Alamos National Laboratory (LANL) [41], which documents a sequence of user operational events within the laboratory's internal computer network, including irregular actions by certain malicious

employees, such as lateral movement and unauthorized access. In this paper, we employ the authentication events, DNS lookup events and network flow events of the LANL dataset. We select all 104 malicious insiders that have ever appeared in red team activity, and randomly select 400 normal users. Our final dataset consists of 4,735,761 authentication events, 619,552 network flows events, and 574,651 DNS lookup events involving 7,825 hosts for 504 users, of which 698 events are malicious attack events.

CERT Dataset: The CERT (Computer Emergency Response Team) dataset [39] is a well-known dataset in the field of insider threat detection, which collects synthetic attacks' activities records and provide both background and malicious actor synthetic data. It is comprised of many types of data, including users' logon, http communication, file operations, emails, and device usage. However, in real-world scenarios, obtaining user email and http communication data can often be challenging due to privacy and security concerns. Therefore, in our experiments, we utilize three files from the latest version of the CERT6.2 dataset, which separately record logon operations, usage of removable storage devices, and file operations. CERT dataset also contains records of real insider threat incidents, providing detailed information on how internal attackers use organizational resources to launch attacks. The final dataset consists of 40,051 operation events from 5 abnormal users and 25 randomly selected normal users, involving 4,400 host nodes. Among these events, there are 161 abnormal events.

TWOS Dataset: The TWOS (The Wolf of SUTD) dataset [42] is collected during a competition held by the Singapore University of Technology and Design in March 2017. It includes six data sources as well as psychological personality questionnaire data. The competition simulates user interactions within and between competing companies over a period of 5 days, including 12 masquerader incidents and 1 traitor incident. In this study, we use logon events, mouse records and keystroke records. The final dataset consists of 1,433,094 operational events involving 32 users and host nodes. Among these events, there are 1,892 malicious events.

B. Experimental Settings

1) **Baselines:** We compare SENTINEL to several state-of-the-art baselines for insider threat detection, such as machine learning-based methods and deep learning-based methods. The machine learning methods applied to compare in this section include two excellent machine learning-based methods of anomaly detection, i.e. Lightweight on-line Detector of Anomalies (LODA) [43] and Local Outlier Factor (LOF) [44], which have shown good performance in insider threats field [1]. The deep learning-based schemes in this section include an LSTM-based sequential language model [26], a graph dynamic detection algorithm named Addgraph [35], an EGAT-based hierarchical APT detection model [5] and two heterogeneous graph learning-based methods, i.e. Log2vec [14] and LMTracker [19].

2) **Metrics:** In this paper, Accuracy, ROC (Receiver Operating Characteristic) Curve and AUC (Area Under the Curve)

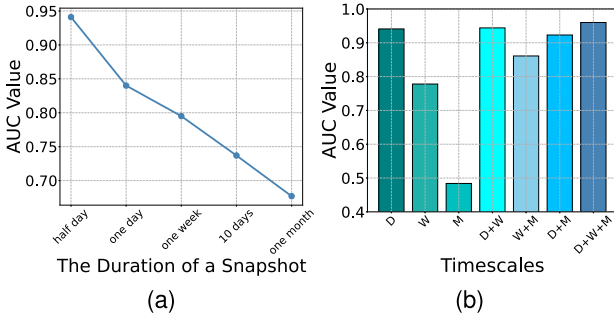


Fig. 6. Performance of SENTINEL under (a) different snapshot duration. (b) different timescales.

metrics are used to measure the performance of insider threat detection. The ROC curve is a plot of the True Positive Rate (TPR) against the False Positive Rate (FPR). The AUC summarizes the ROC curve by measuring the area under the ROC curves. In the insider threat scenarios, relying solely on the accuracy metric can not exactly reflect of the model's detection performance because the majority of samples that need to be detected are normal samples. Considering TPR and FPR jointly, ROC curve and AUC metric can effectively evaluate the performance of model under serious data imbalance.

3) *Parameter Setup*: We divide the three datasets into snapshots with a fixed time duration. For the LANL dataset, we use the first 40 hours of benign activity before the first abnormal event appearing to train the anomaly detector and the rest for testing. The duration of each snapshot is 1 h. Similarly, for the CERT dataset, we use the first 200 days of benign activity for training and the rest for testing. Due to the conspicuous association between employees' regularity and the designated working hours, we set the duration of each snapshot to half a day (12 hours) to distinguish between working hours and off-duty hours. In detail, we divide a day into two distinct time slices: 7:00 a.m. to 7:00 p.m. and 7:00 p.m. to 7:00 a.m.+1. For the TWOS dataset, we select the first day of benign activity for training and the rest for testing. The duration of each graph snapshot is set as 1 h. We set the time window of each graph snapshot stream as 3. We set the learning rate to 0.001, the number of GCN layers to 3, and the regularization weight decay λ to $5e-7$. Dropout is set to 0.2, and the hidden state dimension size is set to 512. The margin γ is set to 0.7. The detection threshold is set as 0.05 for LANL, 0.02 for CERT and 0.05 for TWOS respectively.

4) *Equipment Environment*: We use the TensorFlow library to build SENTINEL, version 2.11.0. We conduct all comparative experiments on a 64-bit CentOS 7.9 operating system. An Nvidia A100 GPUs with 40GB of memory each is used as accelerators.

C. Experiment on Multiple Timescales

To evaluate the performance of SENTINEL under different graph snapshot duration, we conduct experiments on the CERT dataset, which offers an extensive time span for evaluation. We consider five time spans in our evaluation varying from half a day, a day, a week, 10 days to a month (30 days). Fig. 6(a) illustrates the detection performance under different time spans. It is evident that the detection effectiveness improves as the

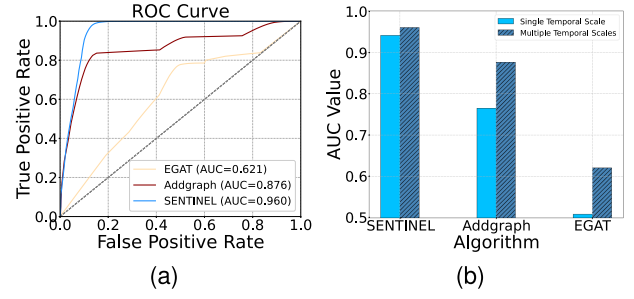


Fig. 7. Performance comparison under multiple timescales. (a) ROC curves. (b) AUC values comparison between single temporal and multiple temporal scales.

duration of snapshot decreases, i.e. the AUC rises from 0.677 to 0.941. A smaller time span implies a shorter log sequence under monitoring, which is more sensitive to the abnormal behaviors. Conversely, a larger time span may potentially overshadow such behavior within the log stream.

In order to explore the impact of different detection timescales on the performance, we change the time interval between snapshots, including day (D), week (W), month (M), day+week (D+W), day+month (D+M), week+month (W+M), and day+week+month (D+W+M). Fig. 6(b) depicts SENTINEL's performance of different timescale combinations. The figure clearly demonstrates that the one-day interval yields the best performance among the single-timescale results. As the increase of time interval, the performance deteriorates significantly due to the difficulty of capturing short-term behavioral patterns. Notably, the model incorporating three different timescales exhibits the highest AUC of 0.960. Leaning the temporal hierarchy with daily, weekly and monthly behavioral patterns enhances the ability to perceive uncertain insider threats. This result indicates that fusing the temporal feature representations of multiple timescales could greatly improve the overall performance.

Afterwards, our investigation focuses on comparing the performance of SENTINEL, AddGraph and EGAT when fusing multi-timescale features. The rest of the baselines are not included in this comparison because they do not utilize feature representations specifically related to multiple timescales. The results presented in Fig. 7 reveal that all the three methods demonstrate a performance improvement compared to the single-timescale results, i.e. an average increase of 0.061 in AUC value. And SENTINEL still outperforms than baselines. In summary, decomposing the temporal effects into multiple scale-specific feature representations proves beneficial for accurately modeling users' behavioral temporal regularity.

D. Experiment on Single Log Source

To further demonstrate the effectiveness of the SENTINEL, we compare its performance to baselines on single data source dataset, which is the validation way that many methods [15], [26], [31], [45], [46] have used. Due to the limited time span,

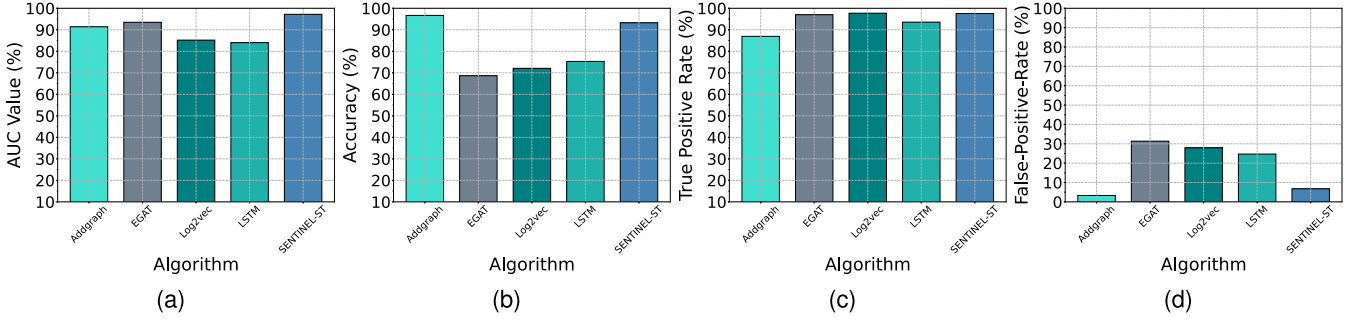


Fig. 8. Performance comparison on single log source for LANL dataset. (a) AUC value. (b) Accuracy. (c) True positive rate. (d) False positive rate.

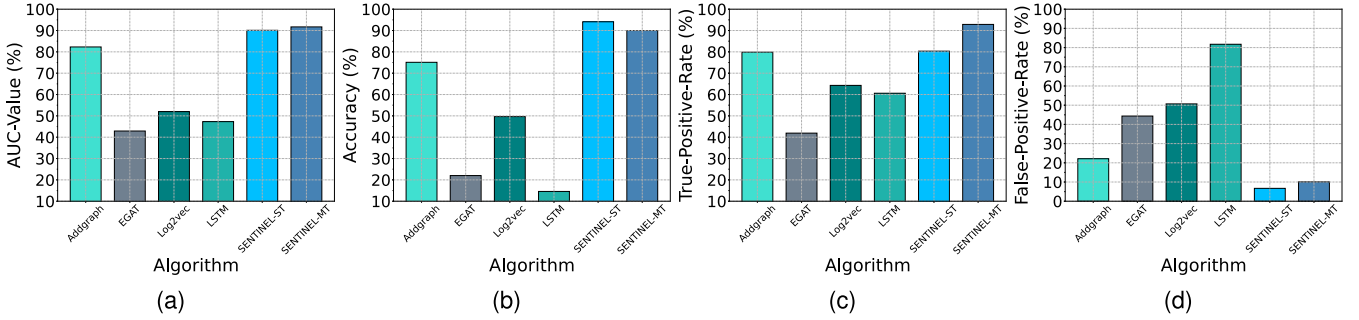


Fig. 9. Performance comparison on single log source for CERT dataset. (a) AUC value. (b) Accuracy. (c) True positive rate. (d) False positive rate.

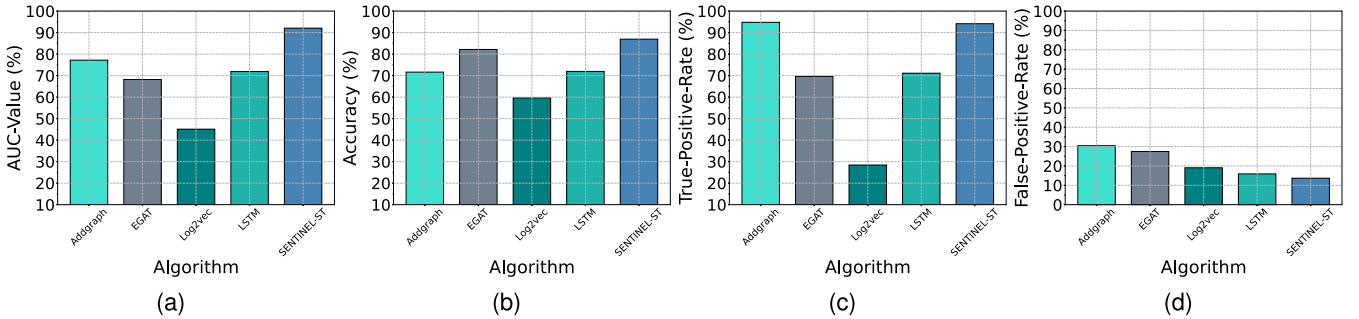


Fig. 10. Performance comparison on single log source for TWOS dataset. (a) AUC value. (b) Accuracy. (c) True positive rate. (d) False positive rate.

we can only evaluate the performance of single-timescale SENTINEL on LANL and TWOS dataset, denoted as SENTINEL-ST. Accordingly, the multi-timescale SENTINEL model is denoted as SENTINEL-MT. In addition, LMTracker, LOF and LODA methods need to collect features involving multiple logs and thus cannot participate in the evaluation on a single data source. For the LANL dataset, we use authentication events, while for the CERT and TWOS dataset, we use users' logon records. In a real-world scenario, security audit experts can leverage this type of log files to identify attackers exhibiting signs of lateral movement or internal employees engaging in covert file exfiltration.

The experimental results are shown in the Figs. 8, 9 and 10. We observe that the SENTINEL outperforms other baseline methods in terms of AUC values for all datasets, i.e. 0.972

for LANL, 0.917 for CERT and 0.921 for TWOS, followed by the Addgraph. The results suggest that the dynamic graph detection methods is sensitive to malicious activities within internal networks. However, since Addgraph hardly considers interactive features, its performance is slightly lower than SENTINEL. The Log2vec shows a good performance on the LANL dataset with the AUC value of 0.852, but underperforms on the CERT and TWOS dataset, as it has high false positive rates with low AUC values of only 0.520 and 0.451. This result can be attributed to the fact that the heterogeneous graph constructed by Log2vec graph rules under the single data source of CERT/TWOS dataset is too sparse, and the random walk based embedding method cannot learn the relationship between logs. Additionally, the graph embedding method employed by Log2vec disregards the attributes of logs, which have an obvious impact on its performance. The

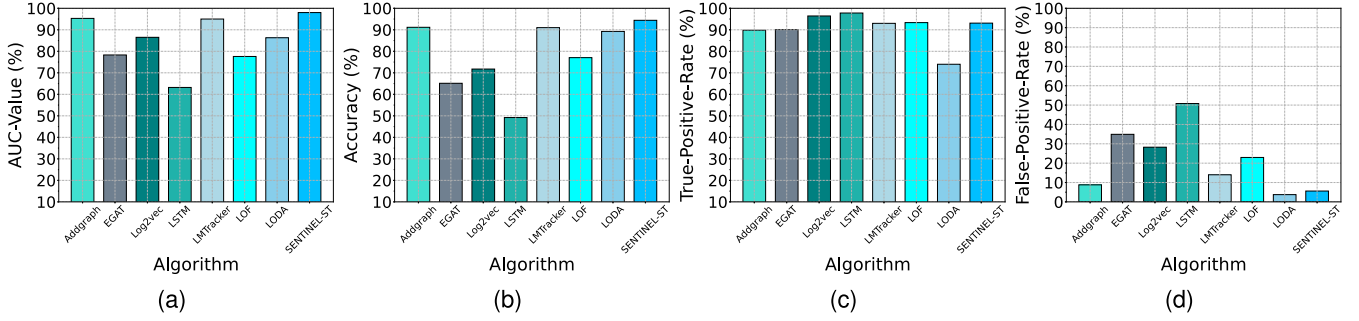


Fig. 11. Performance comparison on multiple log source for LANL dataset. (a) AUC value. (b) Accuracy. (c) True positive rate. (d) False positive rate.

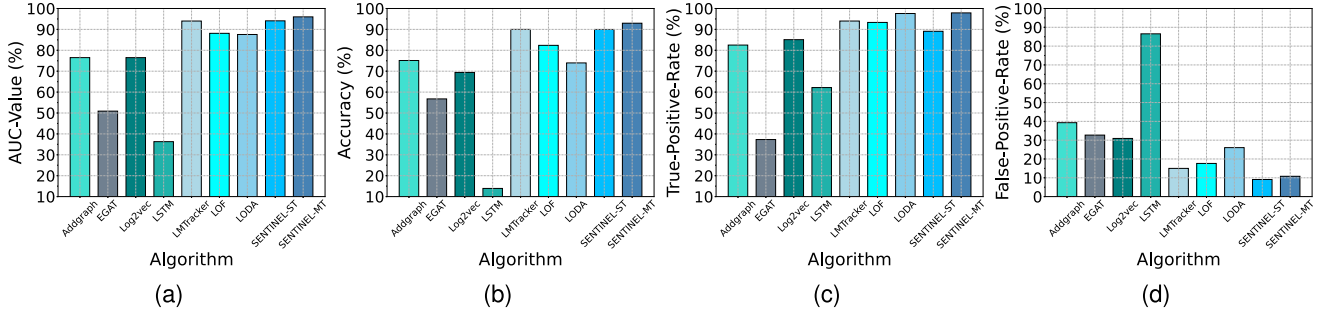


Fig. 12. Performance comparison on multiple log source for CERT dataset. (a) AUC value. (b) Accuracy. (c) True positive rate. (d) False positive rate.

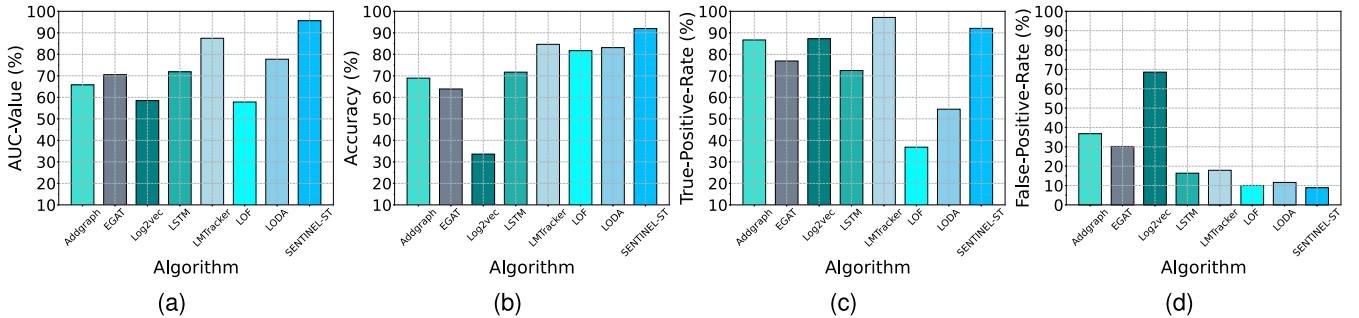


Fig. 13. Performance comparison on multiple log sources for TWOS dataset. (a) AUC value. (b) Accuracy. (c) True positive rate. (d) False positive rate.

EGAT solely relies on edge features and lacks a mechanism to recognize the dynamic changes in the graph structure. This limitation is particularly evident in the results obtained from the CERT dataset, i.e. a low AUC value of 0.473. Although the LSTM model performs well on LANL authentication events, its effectiveness is hindered in the CERT and TWOS dataset. The relatively short length of logon logs results in insufficient context information and adversely impacts its detection performance. The above results indicate that the SENTINEL method can effectively detect insider threats involving logon or authentication events in an internal network.

E. Experiments on Multiple Log Sources

To evaluate the performance of SENTINEL in detecting threats from multi-source data, we conduct experiments on multiple types of log files in LANL, CERT and TWOS datasets. For the LANL dataset, we use authentication events, DNS

events, and network flow events. For the CERT dataset, we use logon records, usage of removable storage devices, and file operation records. For TWOS dataset, we use logon records, mouse records and keystroke records.

Figs. 11, 12 and 13 illustrates the comparison results of SENTINEL and baselines on multiple sources experiments. Table II presents the AUC values comparison between single-source experiments and multi-source experiments. It is observed that SENTINEL consistently achieves the best performance on multi-source experiments, i.e. the largest AUC values of 0.980 on the LANL dataset, 0.960 on the CERT dataset and 0.956 on the TWOS dataset. Compared to the previous single-source results, SENTINEL's multi-source results exhibit an increase in AUC on the three datasets by 0.008, 0.04 and 0.035, respectively. Log2vec shows a notable performance improvement, especially on the CERT dataset where the AUC increases by 0.245. Log2vec utilizes different graph constructing rules to integrate different types of logs into a heterogeneous graph, expanding

TABLE II
AUC COMPARISON BETWEEN SINGLE LOG SOURCE AND MULTIPLE LOG SOURCES

Method	LANL			CERT			TWOS		
	Single Source	Multiple Source	Variation	Single Source	Multiple Source	Variation	Single Source	Multiple Source	Variation
Addgraph	0.914	0.953	+0.039	0.823	0.765	-0.058	0.772	0.658	-0.114
EGAT	0.835	0.791	-0.044	0.429	0.509	+0.080	0.682	0.705	+0.023
Log2vec	0.852	0.865	0.013	0.52	0.765	+0.245	0.451	0.585	+0.134
LSTM	0.840	0.632	-0.208	0.473	0.363	-0.110	0.719	0.712	-0.007
LOF	-	0.776	-	-	0.881	-	-	0.875	-
LODA	-	0.863	-	-	0.876	-	-	0.578	-
LMTracker	-	0.950	-	-	0.910	-	-	0.777	-
SENTINEL	0.972	0.980	+0.008	0.901	0.941	+0.040	0.921	0.956	+0.035

its coverage to monitor more network behaviors and thus can more accurately identify outlier abnormal events. In practice, attackers usually participate in a wide range of threat activities, including but not limited to lateral movement and data theft. Therefore, incorporating a larger number of logs from different sources can improve the accuracy of identifying outliers. Similar method LMTracker, which builds heterogeneous graphs based on meta-paths, also achieves better results because its meta-path rules are more directional. Machine learning-based anomaly detection methods LOF and LODA cannot capture the complex features and interrelations of structural information, thus their performance is often inferior to graph-based methods. However, using multiple log source is also a double-edged sword. For some unstable models like EGAT and LSTM, the use of multiple types of log records may introduce substantial data noise, which may lead to the degradation of performance on the contrary.

F. Comparison of Different Threat Scenarios

In order to explore whether SENTINEL's performance is consistent in different insider threat scenarios, we conduct an analysis of its performance variances within multiple threat scenarios present in the CERT r6.2 dataset. The CERT r6.2 dataset contains four explicit threat scenarios:

- A departing employee, who has never used a removable drive or worked overtime before, logs in after office hours and uploads data to a publicly accessible website (S_1).
- A employee seeking new employment opportunities frequently access job search websites and use a thumb drive to steal confidential information before leaving (S_2).
- A system administrator who is disgruntled with his/her supervisor downloads a keylogger and obtains a list of passwords belonging to various employees within the organization. Subsequently, he/she uses a thumb drive to transfer the password list to the supervisor's machine and attempts to search for the supervisor's password. Ultimately, he/she logs into the supervisor's machine and sends a mass email designed to create panic, resulting in chaos within the organization (S_3).
- An indignant employee logs into other users' computers, searches for files of interest, and sends these files to their personal email address (S_4).

Table III shows the true positive rates and false negatives rate (FNR) of SENTINEL for the above four threat scenarios. As illustrated, SENTINEL performs well on S_1 , S_2 , and S_4 with an

TABLE III
COMPARISON OF DIFFERENT THREAT SCENARIOS

Threat Scenarios	S_1	S_2	S_3	S_4	All
# detected logs /# total	15/15	68/70	7/12	63/64	153/161
TPR	100.00%	97.14%	58.33%	98.44%	95.03%
FNR	0.00%	2.86%	41.67%	1.56%	4.97%

TABLE IV
ABLATION STUDY OF KEY COMPONENTS IN SENTINEL ON THE CERT DATASET

Method	Metrics			
	ACC	TPR	FPR	AUC
w/o EFE	90.15%	85.71%	9.74%	87.60%
w/o SDL	92.79%	85.71%	8.48%	88.78%
w/o TDL	92.95%	98.40%	11.36%	94.98%
w/o MTF	89.97%	89.10%	9.10%	94.10%
Full Model	92.96%	99.90%	10.80%	96.00%

average TPR of 98%, but exhibits poor performance on S_3 where nearly 42% of malicious behaviors undergo undetected. The result reveals that SENTINEL works well in those scenarios (e.g, S_1 , S_2 , S_4) where malicious insiders conduct actions diverging from their routine operations, but may fail in the scenarios (e.g, S_3) where malicious contents is hidden in routine operations. For instance, the insider in S_1 has no prior history of using removable drives or working overtime; in S_2 , there is a sudden spike in the insider's thumb drive device usage; and in S_4 , the insider has never logged into another individual's machine using his/her own credentials before. In contrast, since the crime administrator in S_3 already possesses high authority, his/her historical behavior of frequently logging into other machines may make SENTINEL misclassify the action of logging into the supervisor's machine as normal operation. This suggests that SENTINEL is prone to misjudging malicious actions that closely resemble routine tasks.

G. Ablation Study

We present ablation study in this subsection to evaluate the contribution of each key component on the widely used CERT dataset. The results are presented in Table IV, where EFE represents the edge feature embedding component, SDL and TDL respectively represent the spatial dependency learning component and the temporal dependency learning component of the spatial-temporal graph neural network, and MTF represents the multiple timescale fusion component. We can observe from the

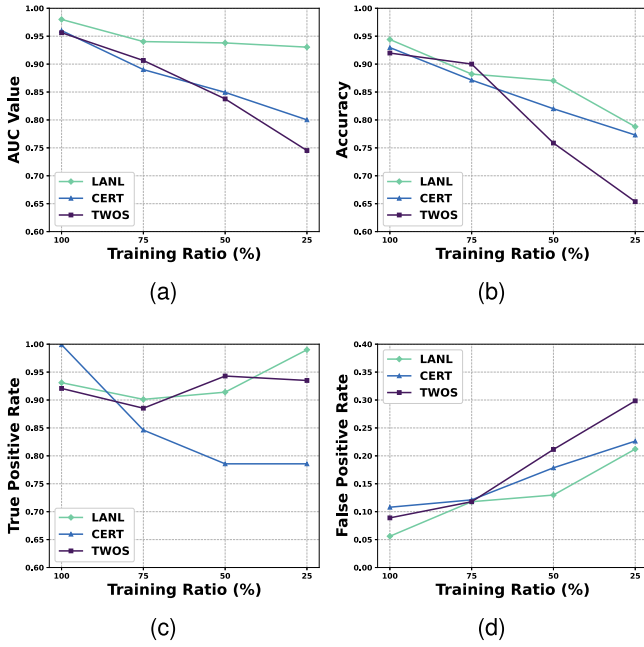


Fig. 14. Performance comparison on varying training sets. (a) AUC value. (b) Accuracy. (c) True positive rate. (d) False positive rate.

table that the w/o EFE model results in the poorest performance, with a decrease in AUC value by 0.084 compared to full model. This result indicates the critical importance of using users' behavioral feature for detecting insider threats. Comparing to the w/o TDL model, the w/o SDL model has a more severe performance degradation due to the absence of spatial topology learning. This result suggests that malicious insider threat behaviors exhibit more significant changes in network topology than in temporal pattern. We also find that the w/o MTF model results in a decrease in AUC value by 0.019, which demonstrates the effectiveness of integrating multi-time scale information.

H. Influence of Varying Training Sets

In order to demonstrate the influence of varying training sets on SENTINEL, we conduct comparative experiments by controlling the proportion of training data. Taking the LANL dataset as an example, a training proportion of 100% indicates that the training set consists of all 40 hours of benign activity prior to the appearance of the first abnormal event whereas 50% means using the first 20 hours as the training set, and so on. Fig. 14 shows the experimental results of SENTINEL across various datasets. We can observe that as the proportion of training data decreases, the overall model performance tends to decline. Specifically, accuracy decreases by an average of 20.8%, the AUC value decreases by an average of 14.6%, and the FPR increases by an average of 2.08 times. This is consistent with our expectations, as a smaller training set causes the model to learn a more prejudiced behavioral baseline, which is unable to cover the diverse range of normal behaviors. Interestingly, we find that SENTINEL's TPR tends to increase on the LANL and TWOS datasets. Our analysis suggests that limited training data might make insider threats more conspicuous, which leads

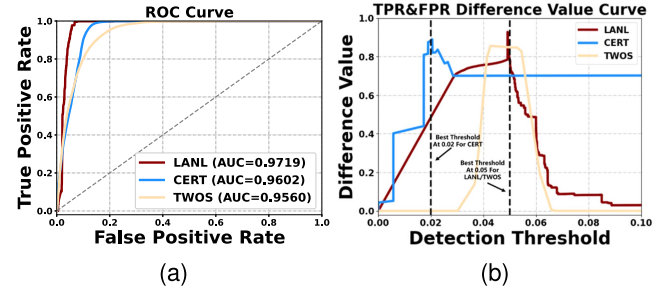


Fig. 15. SENTINEL's (a) ROC curves. (b) TPR&FPR difference curves.

to the detection of more malicious behaviors. However, this benefit comes at the cost of generating more false positives. In general, the experimental results show that a longer training set duration can cover users' normal behavior routines more comprehensively, leading to a more effective trained model.

I. Selection of Detection Threshold

As mentioned in Section IV-F, SENTINEL finally outputs anomaly scores for different user behavior events. To select an appropriate detection threshold, we first plot the ROC curve of SENTINEL on test data. The closer the ROC curve is to the top-left corner (indicating high TPR and low FPR), the better the model's performance. Therefore, we choose the detection threshold corresponding to the point nearest to the upper left corner of the ROC curve, i.e. maximizing (TPR - FPR). Specifically, during the test, setting different detection thresholds th (i.e. samples with anomaly scores greater than th are classified as anomalies) will yield multiple sets of TPR and FPR. These (TPR, FPR) tuples can form a ROC curve. Fig. 15(a) shows the ROC curves of SENTINEL on three datasets. Then, we analyze the differences between the TPR and FPR across various detection thresholds, as illustrated in Fig. 15(b). We select the threshold with the maximum TPR/FPR difference as the decision boundary. For the CERT dataset, the optimal threshold is $th = 0.02$, which results in the largest difference of 0.874. For the LANL and TWOS datasets, the optimal threshold is $th = 0.05$, with the maximum differences reaching 0.892 and 0.824, respectively.

Fig. 16 presents the Kernel Density Estimation (KDE) distributions of the testing samples' anomaly scores for SENTINEL across three datasets. We can observe that the aforementioned selected thresholds effectively differentiate between normal behavior and malicious insider threat behaviors, thereby justifying the appropriateness of these thresholds.

J. Runtime Overhead Analysis

Finally, we investigate the model efficiency of our SENTINEL. We present SENTINEL's training time and resource consumption on three datasets in Table V. As shown in the table, the training time for the LANL dataset is 488 seconds, significantly higher than the other two datasets. This is because the LANL dataset includes 7825 nodes, significantly more than CERT (4400 nodes) and TWOS (32 nodes), and also contains a larger number of logs, resulting in an average of 4246.2 edges per graph snapshot, compared to 145.9 for CERT and 74.6 for

TABLE V
SENTINEL'S TRAINING TIME AND RESOURCE CONSUMPTION

	Training Time	Training GPU Memory Usage	Training GPU Utilization
LANL	488.08s	1117 MB	20.80%
CERT	57.90 s	925 MB	11.20%
TWOS	8.11 s	877 MB	10.90%

TABLE VI
DETECTION COMPUTATIONAL COMPLEXITY OF DIFFERENT COMPARISON METHODS

Method	LOF	LODA	LSTM	Log2vec	Addgraph	EGAT	LMTracker	SENTINEL
Avg.GPU Utilization	-	-	67%	52%	3%	16%	1%	5%
Avg.GPU Memory Usage	-	-	39259MB	5635MB	919MB	9063MB	131MB	922MB
Detection Time	LANL 51.74s	10.89s	1.09h	19.64min	1.29min	2.38min	13.01min	2.15min
	CERT 1.52s	0.31s	52.92s	3.14s	0.92s	1.66 s	6.68min	1.38s
	TWOS 2.47s	1.48s	46.54s	15.66s	0.32s	1.34s	2.24min	0.52s

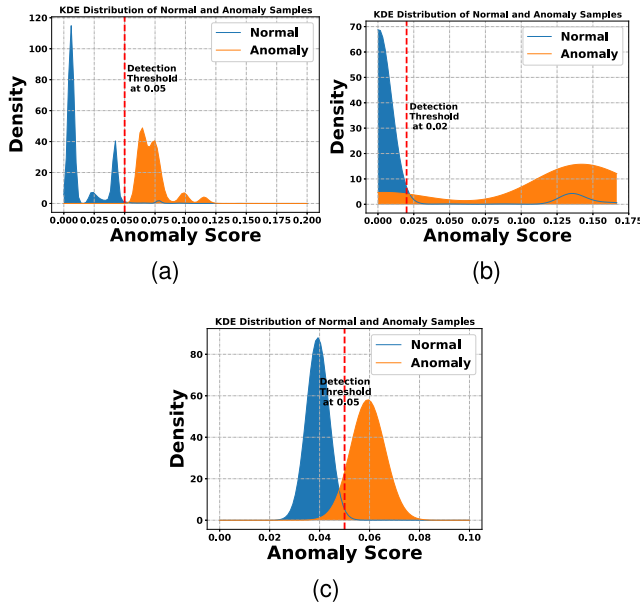


Fig. 16. SENTINEL's KDE distribution of normal and anomaly samples on (a) LANL dataset. (b) CERT dataset. (c) TWOS dataset.

TWOS. Despite the large data scale, most graphs in the LANL dataset are sparse, which means that the number of parameters involved in each update is not excessive. Therefore, the GPU memory usage for LANL training is only 20.7% higher than CERT and 27.4% higher than TWOS, with GPU utilization being approximately twice that of the other datasets. Overall, SENTINEL's training time and resource consumption remain acceptable.

At detection phase, for the two anomaly detection algorithms, i.e. LOF and LODA, we can achieve fast detection speed without GPU acceleration, which benefits from the relatively simple computational architecture of machine learning. However, it should be noted that these two methods rely on pre-extracted complex features, which took more than 90 minutes before testing. In comparison, SENTINEL requires a graph construction process, which takes only less 18 seconds due to the small size of our graph. Table VI shows the average GPU utilization, average GPU memory usage, and time consuming

of different comparative methods. We observe that AddGraph exhibits the least resource occupancy and the fast running speed compared to other methods. By contrast, the detection time and computational overhead are slightly increased for SENTINEL. It is worth noting that the attention-based graph embedding propagation layer of SENTINEL incurs a higher computational cost compared to the adjacent matrix-based graph convolution utilized by AddGraph. Considering the comparison of prediction accuracy between SENTINEL and Addgraph, the additional computational cost could bring positive results via learning the correlation of edge features. Similarly, the attention-based EGAT presents a notable increase in GPU memory usage and utilization due to the high-dimensional node embedding. In Table VI, LSTM and Log2vec exhibit an average GPU utilization of over 50%, which is significantly higher than that of AddGraph and SENTINEL (less than 5%). Furthermore, we also find that the average GPU memory usage of LSTM is the largest, followed by EGAT and Log2vec. In addition, the detection time consuming of SENTINEL, Addgraph and EGAT is far less than the log generation time (mins vs hours), which can meet the needs of real-time insider threat detection. Overall, SENTINEL can achieve a good detection accuracy at the cost of a minor computational burden.

VI. CASE STUDIES

SENTINEL is deployed within an organization's local area network, specifically tailored for detecting insider threats in sensitive government departments and enterprises. By analyzing network interaction behaviors of internal employees and audit logs from terminal hosts, SENTINEL could monitor the threat levels of insiders in real time. If monitored user behavior is deemed highly threatening, such as exhibiting unusual characteristics or logon patterns divergent from typical activities, the security operation center will initiate appropriate responses. Below, we illustrate two typical insider threat scenarios, i.e. malicious insiders and compromised insiders.

- *Malicious Insiders*: User 1 typically logs into his/her host between 8:30 and 9:00 AM, and logs out between 6:30 and 7:00 PM. However, this user frequently logs into other users' computers around 12:00 noon and 6:00 PM, searches for files of interest, and subsequently emails these files to

personal addresses. This unusual logon pattern triggers an alert from SENTINEL.

- *Compromised Insiders*: User 2 steals user 3's credentials through a covertly installed keylogger. Subsequently, he/she utilizes these credentials to remotely access the database server and implant a backdoor, aiming to monitor information flow and steal commercial secrets. Since the Service Principal Name of this server is registered in Active Directory, user 3 generally accesses the server through the Kerberos protocol to retrieve information. However, user 2's backdoor software operates outside Active Directory and thus he/she is compelled to use NTLM protocol for authentication. SENTINEL identifies this shift in user operation attributes, and signals a potential security risk.

VII. CONCLUSION AND FUTURE WORK

In this paper, we propose a novel method SENTINEL, which utilizes an edge-enhanced spatio-temporal graph neural network for detecting insider threats. In SENTINEL, user's behavior interactions within the internal network are formed as a graph. We subsequently employ an ST-GNN to learn the spatio-temporal operation regularities of users. SENTINEL adopts a semi-supervised strategy that requires only normal behavior log data for training, which enhances the flexibility of deployment. Evaluation on the LANL/CERT/TWOS datasets demonstrates that SENTINEL outperforms state-of-the-art approaches in terms of AUC and ROC curve. To address the problem that SENTINEL is prone to misjudging malicious actions that closely resemble routine operation, we will integrate additional behavioral indicators, such as employees' mental health status or social records for insider threat detection in the future. Moreover, we will focus on automating the reconstruction of the attack scenarios for insider threat based on threat alerts. This will provide a clear understanding of the attack strategy and intentions.

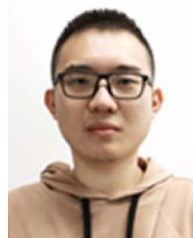
REFERENCES

- [1] D. C. Le and N. Zincir-Heywood, "Anomaly detection for insider threats using unsupervised ensembles," *IEEE Trans. Netw. Service Manag.*, vol. 18, no. 2, pp. 1152–1164, Jun. 2021.
- [2] Proofpoint, "2022 ponemon cost of insider threats global report," 2022. [Online]. Available: <https://www.proofpoint.com/us/resources/threat-reports/cost-of-insider-threats>
- [3] N. F. Syed, S. W. Shah, A. Shaghghi, A. Anwar, Z. Baig, and R. Doss, "Zero trust architecture (ZTA): A comprehensive survey," *IEEE Access*, vol. 10, pp. 57143–57179, 2022.
- [4] D. C. Le, N. Zincir-Heywood, and M. I. Heywood, "Analyzing data granularity levels for insider threat detection using machine learning," *IEEE Trans. Netw. Service Manag.*, vol. 17, no. 1, pp. 30–44, Mar. 2020.
- [5] Z. Li, X. Cheng, L. Sun, J. Zhang, and B. Chen, "A hierarchical approach for advanced persistent threat detection with attention-based graph neural networks," *Secur. Commun. Netw.*, vol. 2021, 2021, Art. no. 9961342.
- [6] M. Villarreal-Vasquez, G. M. Howard, S. Dube, and B. Bhargava, "Hunting for insider threats using LSTM-based anomaly detection," *IEEE Trans. Dependable Secur. Comput.*, vol. 20, no. 1, pp. 451–462, Jan./Feb. 2023.
- [7] L. Liu, O. De Vel, Q.-L. Han, J. Zhang, and Y. Xiang, "Detecting and preventing cyber insider threats: A survey," *IEEE Commun. Surveys Tut.*, vol. 20, no. 2, pp. 1397–1417, Second Quarter 2018.
- [8] A. Sah et al., "A new architecture for managing enterprise log data," in *Proc. Conf. Syst. Admin.*, 2002, pp. 121–132.
- [9] M. Hanley and J. Montelibano, "Insider threat control: Using centralized logging to detect data exfiltration near insider termination," DTIC, Fort Belvoir, VA, USA, Tech. Rep. 024, 2011.
- [10] S. M. Milajerdi, R. Gjomemo, B. Eshete, R. Sekar, and V. Venkatakrishnan, "HOLMES: Real-time APT detection through correlation of suspicious information flows," in *Proc. IEEE Symp. Secur. Privacy*, 2019, pp. 1137–1152.
- [11] Y. Shen, E. Mariconti, P. A. Vervier, and G. Stringhini, "Tiresias: Predicting security events through deep learning," in *Proc. ACM Conf. Comput. Commun. Secur.*, 2018, pp. 592–605.
- [12] P. Chattopadhyay, L. Wang, and Y.-P. Tan, "Scenario-based insider threat detection from cyber activities," *IEEE Trans. Comput. Soc. Syst.*, vol. 5, no. 3, pp. 660–675, Sep. 2018.
- [13] S. Lei, C. Xia, Z. Li, X. Li, and T. Wang, "HNN: A novel model to study the intrusion detection based on multi-feature correlation and temporal-spatial analysis," *IEEE Trans. Netw. Sci. Eng.*, vol. 8, no. 4, pp. 3257–3274, Oct.–Dec. 2021.
- [14] F. Liu, Y. Wen, D. Zhang, X. Jiang, X. Xing, and D. Meng, "Log2vec: A heterogeneous graph embedding based approach for detecting cyber threats within enterprise," in *Proc. ACM Conf. Comput. Commun. Secur.*, 2019, pp. 1777–1794.
- [15] H. Bian, T. Bai, M. A. Salahuddin, N. Limam, A. Abou Daya, and R. Boutaba, "Uncovering lateral movement using authentication logs," *IEEE Trans. Netw. Serv. Manag.*, vol. 18, no. 1, pp. 1049–1063, Mar. 2021.
- [16] J. Xiao, L. Yang, F. Zhong, X. Wang, H. Chen, and D. Li, "Robust anomaly-based insider threat detection using graph neural network," *IEEE Trans. Netw. Service Manag.*, vol. 20, no. 3, pp. 3717–3733, Sep. 2022.
- [17] X. Li et al., "A high accuracy and adaptive anomaly detection model with dual-domain graph convolutional network for insider threat detection," *IEEE Trans. Inf. Forensic Secur.*, vol. 18, pp. 1638–1652, 2023.
- [18] F. Liu, Y. Wen, Y. Wu, S. Liang, X. Jiang, and D. Meng, "ML-Tracer: Malicious logins detection system via graph neural network," in *Proc. IEEE Int. Conf. Trust, Secur. Privacy Comput. Commun.*, 2020, pp. 715–726.
- [19] Y. Fang, C. Wang, Z. Fang, and C. Huang, "LMTracker: Lateral movement path detection based on heterogeneous graph embedding," *Neurocomputing*, vol. 474, pp. 37–47, 2022.
- [20] Z. Wang, J. Chen, and H. Chen, "EGAT: Edge-featured graph attention network," in *Proc. 30th Int. Conf. Artif. Neural Netw.*, Springer, 2021, pp. 253–264.
- [21] X. Zhang et al., "Traffic flow forecasting with spatial-temporal graph diffusion network," in *Proc. AAAI Conf. Artif. Intell.*, 2021, pp. 15008–15015.
- [22] A. Ambre and N. M. Shekhar, "Insider threat detection using log analysis and event correlation," *Procedia Comput. Sci.*, vol. 45, pp. 436–445, 2015.
- [23] MITRE, "Adversarial tactics, techniques and common knowledge," (n.d.). 2013. [Online]. Available: <https://attack.mitre.org>
- [24] T. E. Senator et al., "Detecting insider threats in a real corporate database of computer usage activity," in *Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Min.*, 2013, pp. 1393–1401.
- [25] W. Meng, K.-K. R. Choo, S. Furnell, A. V. Vasilakos, and C. W. Probst, "Towards Bayesian-based trust management for insider attacks in healthcare software-defined networks," *IEEE Trans. Netw. Service Manag.*, vol. 15, no. 2, pp. 761–773, Jun. 2018.
- [26] A. R. Tuor, R. Baerwolf, N. Knowles, B. Hutchinson, N. Nichols, and R. Jasper, "Recurrent neural network language models for open vocabulary event-level cyber anomaly detection," in *Proc. AAAI Conf. Artif. Intell.*, 2018, pp. 285–293.
- [27] K. D. Randive and M. Ramasundaram, "MWCapsNet: A novel multi-level wavelet capsule network for insider threat detection using image representations," *Neurocomputing*, vol. 553, 2023, Art. no. 126588.
- [28] W. Eberle, J. Graves, and L. Holder, "Insider threat detection using a graph-based approach," *J. Appl. Secur. Res.*, vol. 6, no. 1, pp. 32–81, 2010.
- [29] B. Perozzi, R. Al-Rfou, and S. Skiena, "Deepwalk: Online learning of social representations," in *Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Min.*, pp. 701–710, 2014.
- [30] Y. Zhang, C. Yang, K. Huang, and Y. Li, "Intrusion detection of industrial Internet-of-Things based on reconstructed graph neural networks," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 5, pp. 2894–2905, Sep./Oct. 2023.
- [31] B. Bowman, C. Laprade, Y. Ji, and H. H. Huang, "Detecting lateral movement in enterprise computer networks with unsupervised graph AI," in *Proc. Int. Symp. Res. Attacks, Intrusions Defenses*, 2020, pp. 257–268.
- [32] G. Ho et al., "Hopper: Modeling and detecting lateral movement," in *Proc. USENIX Secur. Symp.*, 2021, pp. 3093–3110.
- [33] C. Wang and H. Zhu, "Wrongdoing monitor: A graph-based behavioral anomaly detection in cyber security," *IEEE Trans. Inf. Forensic Secur.*, vol. 17, pp. 2703–2718, 2022.

- [34] T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado, and J. Dean, "Distributed representations of words and phrases and their compositionality," in *Proc. Adv. Neural Inf. Proces. Syst.*, 2013, pp. 3111–3119.
- [35] L. Zheng, Z. Li, J. Li, Z. Li, and J. Gao, "AddGraph: Anomaly detection in dynamic graph using attention-based temporal GCN," in *Proc. Int. Joint Conf. Artif. Intell.*, 2019, pp. 4419–4425.
- [36] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. Int. Conf. Learn. Representations*, 2017, pp. 1–12.
- [37] Y. Kwon et al., "MCI: Modeling-based causality inference in audit logging for attack investigation," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, vol. 2, p. 4, 2018.
- [38] Q. Cui, S. Wu, Y. Huang, and L. Wang, "A hierarchical contextual attention-based network for sequential recommendation," *Neurocomputing*, vol. 358, pp. 141–149, 2019.
- [39] J. Glasser and B. Lindauer, "Bridging the gap: A pragmatic approach to generating insider threat data," in *Proc. IEEE CS Secur. Privacy Workshops*, 2013, pp. 98–104.
- [40] Z. Wang, J. Zhang, J. Feng, and Z. Chen, "Knowledge graph embedding by translating on hyperplanes," in *Proc. AAAI Conf. Artif. Intell.*, 2014, pp. 1112–1119.
- [41] A. D. Kent, "Comprehensive, Multi-source cyber-security events," Los Alamos Nat. Lab., Los Alamos, NM, USA, 2015, doi: [10.17021/1179829](https://doi.org/10.17021/1179829).
- [42] A. Harilal, F. Toffalini, J. Castellanos, J. Guarnizo, I. Homoliak, and M. Ochoa, "TWOS: A dataset of malicious insider threat behavior based on a gamified competition," in *Proc. Int. Workshop Manag. Insider Secur. Threats*, 2017, pp. 45–56.
- [43] T. Pevny, "Loda: Lightweight on-line detector of anomalies," *Mach. Learn.*, vol. 102, pp. 275–304, 2016.
- [44] M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, "LOF: Identifying density-based local outliers," in *Proc. ACM SIGMOD Int. Conf. Manag. Data*, 2000, pp. 93–104.
- [45] R. Paudel and H. H. Huang, "Pikachu: Temporal walk based dynamic graph embedding for network anomaly detection," in *Proc. IEEE/IFIP Netw. Oper. Manag. Symp.*, 2022, pp. 1–7.
- [46] Y. Yuan, S. S. Adhatarao, M. Lin, Y. Yuan, Z. Liu, and X. Fu, "ADA: Adaptive deep log anomaly detector," in *Proc. IEEE Conf. Comput. Commun.*, 2020, pp. 2449–2458.



Siyang Chen (Student Member, IEEE) received the B.S. degree in 2019 from the University of Science and Technology of China (USTC), Hefei, China, where he is currently working toward the Ph.D. degree with the School of Information Science and Technology, USTC, affiliated with the Laboratory for Future Network. His research interests include network security traffic analysis, and website fingerprinting.



Yuanyi Ma (Student Member, IEEE) received the B.S. degree in 2018 from the University of Science and Technology of China, Hefei, China, where he is currently working toward the Ph.D. degree with the School of Information Science and Technology. His research interests include future network, network optimization, and network security.



Huasen He (Member, IEEE) received the B.S. degree in automation from the University of Science and Technology of China (USTC), Hefei, China, in 2013, and the M.S. degree in signal processing and communications and the Ph.D. degree in digital communications from the University of Edinburgh, Edinburgh, U.K., in 2014 and 2018, respectively. He is currently an Associate Research Fellow with the School of Information Science and Technology, USTC. His research interests include future networks, network modeling, and optimization.



Fengrui Xiao (Student Member, IEEE) received the B.E. degree in information science and technology in 2019 from the University of Science and Technology of China (USTC), Hefei, China, where he is currently working toward the Ph.D. degree. His research interests include encrypted traffic classification, insider threats, and multi-step attack.



Jian Yang (Senior Member, IEEE) received the B.S. and Ph.D. degrees from the University of Science and Technology of China (USTC), Hefei, China, in 2001 and 2006, respectively. He is currently a Professor in the School of Information Science and Technology, USTC. His research interests include future network, distributed system design, modeling and optimization, multimedia over wired and wireless and stochastic optimization. Dr. Yang was the recipient of the Lu Jia-Xi Young Talent Award from Chinese Academy of Sciences in 2009.



Shuangwu Chen (Member, IEEE) received the B.S. and Ph.D. degrees from the University of Science and Technology of China (USTC), Hefei, China, in 2011 and 2016, respectively. He is currently an Associate Professor with the Department of Automation, USTC. His research interests include multimedia communications, future network, and stochastic optimization.